

DLX Processor Design and Implementation

Yesshake Zinabu Tadesse

September, 2026

Contents

1	Instruction Set Architecture	3
1.1	Instruction Encoding	3
1.2	Opcode and Function Encoding	4
1.3	Implemented Instruction Set	5
2	Processor Architecture	6
2.1	Top Level Pipeline Organization	6
2.2	Instruction Fetch Stage	7
2.3	Instruction Decode Stage	8
2.3.1	Instruction Fields, Register File, and Immediate Generation	8
2.3.2	Decode Side Forwarding	9
2.3.3	Scoreboard Based Hazard Detection	9
2.3.4	Branch and Jump Resolution	10
2.4	Execute Stage	10
2.4.1	Two stage Radix-4 Booth–Dadda Multiplier	11
2.4.2	P4 Adder	11
2.5	Memory Stage	11
2.5.1	External Memory Interface	12
2.6	Writeback Stage	12
3	Functional Verification	13
3.1	Verification Environment	13
3.2	Directed Test Programs	13
3.3	RTL Verification Result	13
4	Synthesis Flow and Methodology	14
4.1	Timing Driven Synthesis Exploration	14
4.2	Synthesis Constraint and Achieved Operating Period	14
4.3	Clock Sweep Procedure	15
4.4	Clock Gating Design Space Exploration	15
4.5	Post-Synthesis Functional Simulation	15
4.6	Activity Aware Power Analysis	15
4.7	Valid Mapping Results	15
4.8	Final Mapping Selection	17
4.9	Methodological Summary	17
5	Physical Design	18
5.1	Implementation Flow	18
5.2	Timing Analysis	20
5.3	Post Route Results	20

1 Instruction Set Architecture

The processor implements a 32 bit DLX instruction set with 32 general purpose registers and 32 bit width instructions. Register R0 is hardwired to zero. JAL and JALR use R31 as the link register and write the return address $PC + 4$.

1.1 Instruction Encoding

The instruction word is decoded using three encoding families: register type (R-type), immediate type (I-type), and jump type (J-type). All instructions place the 6 bit opcode in bits [31 : 26]. For R-type instructions the opcode is zero and the operation is selected by a 6 bit function field in bits [5 : 0]. I-type instructions use the opcode and carry a 16 bit immediate. J and JAL use 26 bits as a signed, PC relative displacement.

The bit layouts implemented by the RTL are shown below. The names correspond directly to the fields extracted in the Decode stage.

Bits	31:26	25:21	20:16	15:11	10:6	5:0
Field	opcode	rs1	rs2	rd	reserved	func
Width	6	5	5	5	5	6

Table 1: R-type instruction encoding.

For every implemented R-type instruction, `opcode[31:26]` is 000000. The source register addresses are taken from bits [25 : 21] and [20 : 16], the destination register from bits [15 : 11], and the final operation is selected by `func[5:0]`. Bits [10 : 6] are not used by the current decoder and are therefore reserved in this implementation. This also means that the register shift instructions use the normal register operands; there is no separately decoded R-type shift-amount field.

Bits	31:26	25:21	20:16	15:0
Field	opcode	rs1	rd/rs2	imm16
Width	6	5	5	16

Table 2: I-type instruction encoding.

The meaning of bits [20 : 16] depends on the instruction class. For arithmetic, logical, shift, comparison, and load instructions, this field is the destination register. For stores, it identifies the second source register, which provides the data written to memory. Conditional branches only consume `rs1`; the [20 : 16] field has no branch semantic role in the current implementation. The 16 bit immediate is interpreted according to the decoded operation: arithmetic, address-generation, branch, and signed-comparison instructions use sign extension; logical and unsigned-comparison immediates use zero extension; LHI places the immediate in bits [31 : 16] of the result.

Bits	31:26	25:0
Field	opcode	imm26
Width	6	26

Table 3: J-type instruction encoding used by J and JAL.

For J and JAL, `imm26` is sign extended to 32 bits, shifted left by two, and added to $PC + 4$:

$$PC_{target} = PC + 4 + (\text{signext}(\text{imm26}) \ll 2). \quad (1)$$

Conditional branches use the same target-generation rule with the 16 bit immediate:

$$PC_{target} = PC + 4 + (\text{signext}(\text{imm16}) \ll 2). \quad (2)$$

JR and JALR are encoded using their own opcodes rather than the R-type function field. Their jump target is read from `rs1[25:21]`; no immediate target is used. JAL and JALR both write $PC + 4$ to R31. The destination is not encoded in the instruction: the Decode stage explicitly forces the destination register address to 11111 for these two link instructions.

1.2 Opcode and Function Encoding

Table 4 gives the opcode assignments that are actually decoded by the control unit. Hexadecimal values are included to make machine-code inspection and assembler output easier to read.

Instruction	Binary	Hex	Instruction	Binary	Hex
R-type	000000	0x00	BLTZ	000001	0x01
J	000010	0x02	JAL	000011	0x03
BEQZ	000100	0x04	BNEZ	000101	0x05
BLEZ	000110	0x06	BGTZ	000111	0x07
ADDI	001000	0x08	ADDUI	001001	0x09
SUBI	001010	0x0A	SUBUI	001011	0x0B
ANDI	001100	0x0C	ORI	001101	0x0D
XORI	001110	0x0E	LHI	001111	0x0F
BGEZ	010000	0x10	JR	010010	0x12
JALR	010011	0x13	SLLI	010100	0x14
NOP	010101	0x15	SRLI	010110	0x16
SRAI	010111	0x17	SEQI	011000	0x18
SNEI	011001	0x19	SLTI	011010	0x1A
SGTI	011011	0x1B	SLEI	011100	0x1C
SGEI	011101	0x1D	LB	100000	0x20
LW	100011	0x23	LBU	100100	0x24
SB	101000	0x28	SW	101011	0x2B
SLTUI	111010	0x3A	SGTUI	111011	0x3B
SLEUI	111100	0x3C	SGEUI	111101	0x3D

Table 4: Implemented opcode assignments.

R-type instructions share opcode 0x00. Their operation is selected by the function field shown in Table 5.

The package also defines opcode constants for LH (0x21), LHU (0x25), and SH (0x29). These values are not handled by the current control-unit decode logic, so those instructions are not part of the implemented ISA.

Instruction	Binary	Hex	Instruction	Binary	Hex
SLL	000100	0x04	SRL	000110	0x06
SRA	000111	0x07	ADD	100000	0x20
ADDU	100001	0x21	SUB	100010	0x22
SUBU	100011	0x23	AND	100100	0x24
OR	100101	0x25	XOR	100110	0x26
SEQ	101000	0x28	SNE	101001	0x29
SLT	101010	0x2A	SGT	101011	0x2B
SLE	101100	0x2C	SGE	101101	0x2D
MULT	101110	0x2E	SLTU	111010	0x3A
SGTU	111011	0x3B	SLEU	111100	0x3C
SGEU	111101	0x3D			

Table 5: Implemented R-type function-field assignments.

1.3 Implemented Instruction Set

The implemented instruction set is summarized in Table 6. Signed and unsigned variants of addition and subtraction use the same 32 bit two’s-complement arithmetic datapath; the signedness distinction is relevant to the ISA encoding but does not require a different addition circuit. Signed and unsigned relational instructions are handled separately by the comparison logic.

Class	Instructions
Register arithmetic/logic	ADD, ADDU, SUB, SUBU, AND, OR, XOR
Register shifts	SLL, SRL, SRA
Register comparisons	SEQ, SNE, SLT, SGT, SLE, SGE, SLTU, SGTU, SLEU, SGEU
Multiplication	MULT
Immediate arithmetic/logic	ADDI, ADDUI, SUBI, SUBUI, ANDI, ORI, XORI, LHI
Immediate shifts	SLLI, SRLI, SRAI
Immediate comparisons	SEI, SNEI, SLTI, SGTI, SLEI, SGEI, SLTUI, SGTUI, SLEUI, SGEUI
Conditional branches	BLTZ, BEQZ, BNEZ, BLEZ, BGTZ, BGEZ
Jumps and links	J, JAL, JR, JALR
Loads	LB, LBU, LW
Stores	SB, SW
Other	NOP

Table 6: Implemented DLX instruction set.

2 Processor Architecture

The processor is organized as a five stage in-order pipeline: Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory (MEM), and Writeback (WB). The basic pipeline is conventional, but several decisions were made to reduce control and data-hazard penalties while keeping five visible processor stages. Control-flow instructions are resolved in Decode, Fetch contains a Branch Target Buffer (BTB), forwarding is distributed across ID, EX, and MEM, and a scoreboard-based hazard unit stalls only when a value cannot be supplied by the stage in which the younger instruction needs it.

Multiplication is handled by a two stage pipelined Booth–Dadda datapath inside the execution path rather than by extending the visible processor pipeline. External instruction and data memories use explicit issue/ready handshakes, so the pipeline can tolerate memory latency without assuming a fixed response time.

2.1 Top Level Pipeline Organization

The stage boundaries are formed by IF/ID, ID/EX, EX/MEM, and MEM/WB pipeline registers. Alongside the datapath, the decoded control word is pipelined toward the stage in which each control signal is required. Valid bits are carried with the pipeline state and are used to distinguish real instructions from bubbles. This is important for hazard stalls, branch correction, delayed multiplication results, and memory backpressure.

At the top level, the forwarding unit observes producer and consumer register addresses across the pipeline and generates the selection signals used in ID, EX, and MEM. The hazard unit is located on the Decode side and controls whether the current IF/ID instruction may issue. The branch unit also operates from ID and returns both the resolved control-flow information and the correction signal to Fetch. The external memory interface is kept separate from the individual pipeline stages and translates the processor-side instruction/data requests to the external issue/ready protocol.

Stage	Main functions
IF	PC generation, sequential fetch, BTB lookup, branch-correction redirection, instruction memory handshake, and IF/ID registration.
ID	Instruction decode, register-file access, immediate generation, Decode forwarding, RAW-hazard detection, scoreboard update, and branch/jump resolution.
EX	Operand forwarding, arithmetic, comparison, logic, shifting, link/immediate pass-through, and pipelined multiplication.
MEM	Load/store handling, address alignment, byte selection/extension, store-data forwarding, memory access, and writeback-data selection.
WB	Final register-file writeback and WB forwarding.

Table 7: Main responsibility of each pipeline stage.

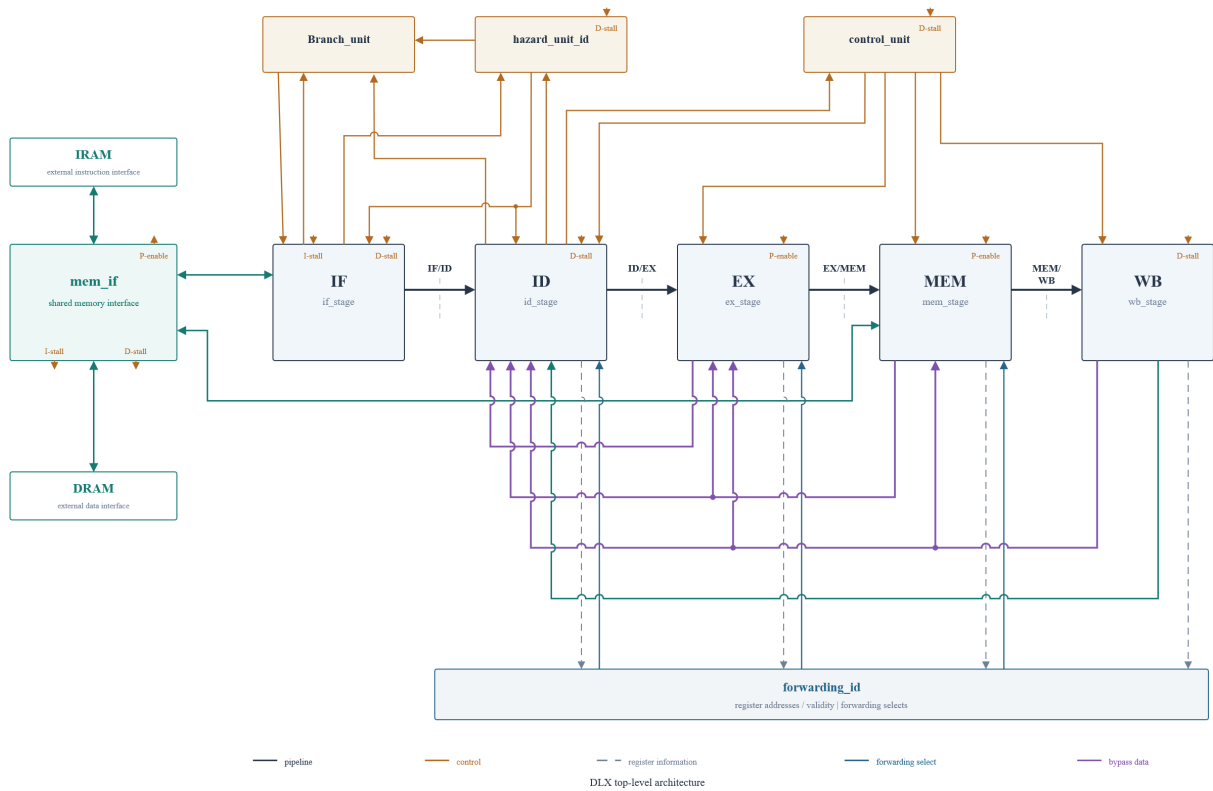


Figure 1: DLX top level block diagram

The responsibility of each stage is summarized in Table 7.

2.2 Instruction Fetch Stage

The Fetch stage maintains the program counter and supplies the IF/ID pipeline register. Under normal sequential execution the next address is $PC + 4$. The next-PC mux can instead select a BTB target or a correction address returned by the Decode stage branch unit.

The BTB contains 16 direct mapped entries. Address bits $[5 : 2]$ select the entry, while the upper address bits are used as the tag. Each entry stores a valid bit, a target address, and a two bit saturating counter. A matching entry is predicted taken when the most significant bit of its counter is set. Jumps are trained as taken; conditional branches update the counter in the direction of the resolved outcome.

For every accepted fetch, IF registers the instruction, the current PC, $PC + 4$, the predicted next address, and a validity bit. The predicted next address is later compared with the actual next address in Decode. A mismatch causes the wrong-path instruction to be invalidated and redirects Fetch to the resolved address.

Fetch can be held by a Decode hazard, an unavailable instruction memory response, or a data memory stall that freezes the complete pipeline. Branch correction is applied once the pipeline is able to advance.

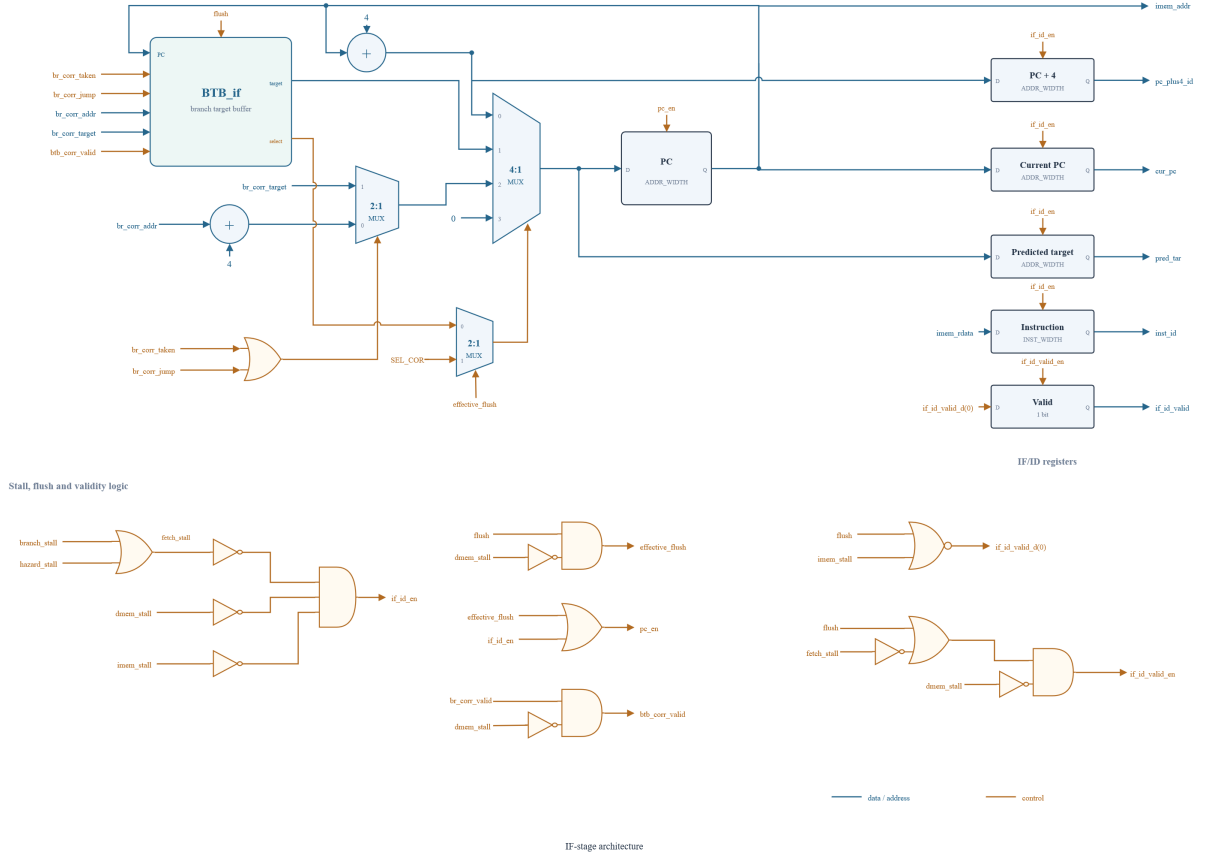


Figure 2: Instruction fetch stage diagram

2.3 Instruction Decode Stage

Decode is the control-heavy stage of the processor. It performs instruction field extraction, register-file access, immediate generation, control decoding, Decode-side forwarding, hazard detection, and branch/jump resolution. Resolving control flow here keeps the branch correction penalty small, but it also places register dependency checks, forwarding selection, branch evaluation, target generation, and Fetch redirection in the same stage-to stage control path.

2.3.1 Instruction Fields, Register File, and Immediate Generation

The opcode is taken from bits [31 : 26] and the R-type function field from bits [5 : 0]. RS1 and RS2 are read from bits [25 : 21] and [20 : 16]. The R-type destination is bits [15 : 11], while I-type instructions use bits [20 : 16] as the destination. JAL and JALR override the decoded destination and select R31.

The register file provides two combinational read ports and one synchronous write port. Writes to R0 are suppressed. A same-cycle WB bypass is also present: when the register written in WB is read by Decode in that cycle, the incoming WB data is returned instead of the old array value.

The immediate-generation unit receives the low 26 instruction bits and applies the extension selected by the decoder. It supports sign extension of 16 bit immediates, zero extension of 16 bit immediates, sign extension of 26 bit jump immediates, and the upper-immediate transformation used by LHI.

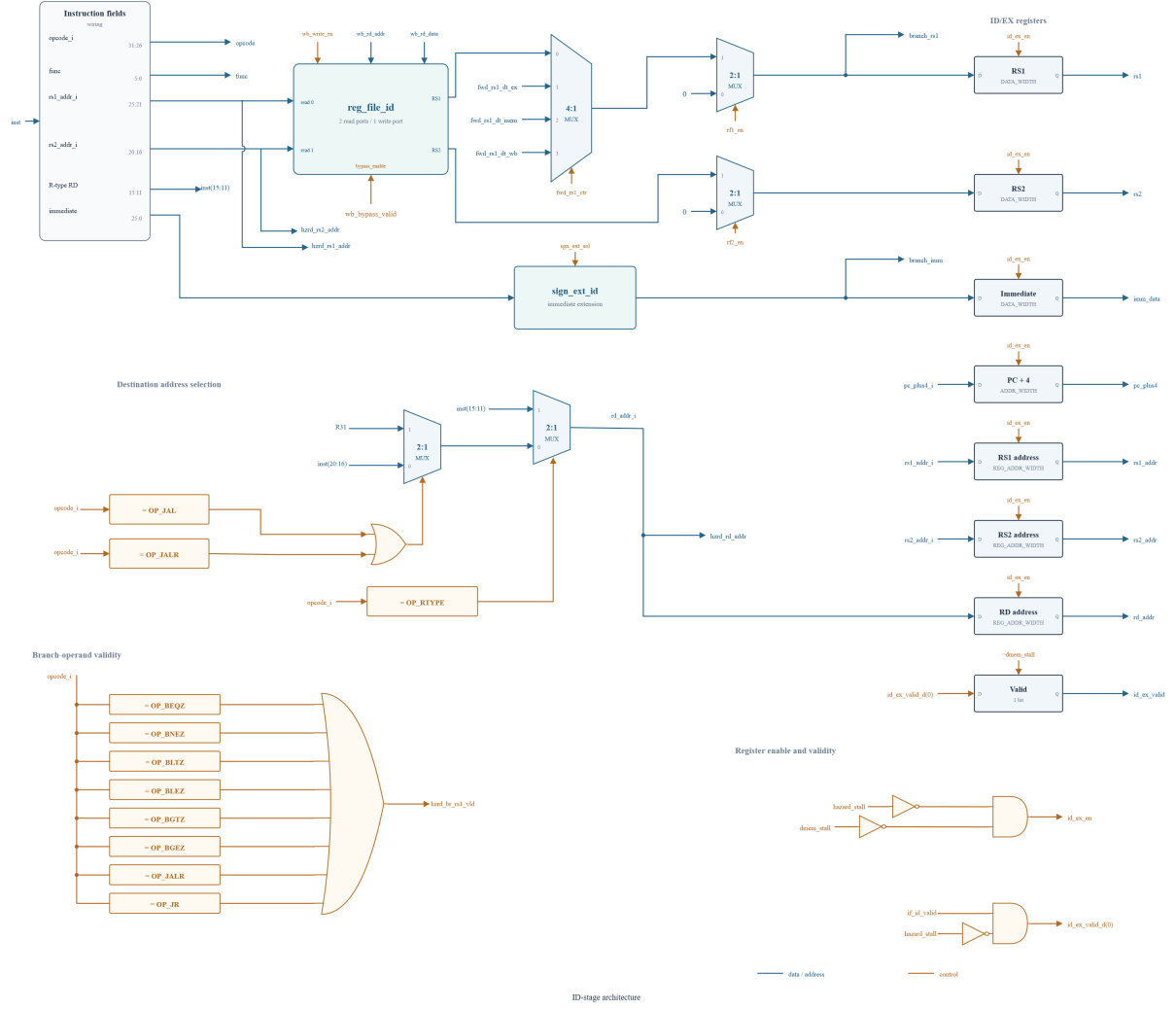


Figure 3: Instruction Decode Stage Block Diagram

2.3.2 Decode Side Forwarding

Branches and register indirect jumps consume RS1 in Decode, before an older producer may have written its result to the register file. A dedicated Decode forwarding mux therefore selects the youngest matching value with the priority

$$\text{EX} > \text{MEM} > \text{WB} > \text{Register File.} \quad (3)$$

Only RS1 requires a Decode-side bypass because it is the operand used by the branch/jump resolution logic. Ordinary execution operands are forwarded later, at the EX stage inputs.

The complete forwarding organization is distributed across the pipeline as shown in Table 8.

2.3.3 Scoreboard Based Hazard Detection

The hazard unit uses a per register scoreboard to track how many stage steps remain before an ongoing destination can satisfy a younger consumer. The important point is that the unit distinguishes both *when a result is produced* and *when the younger instruction consumes it*. A dependency is therefore not automatically a stall.

Consumers are classified as ID, EX, or MEM consumers. Branches, JR, and JALR consume RS1 in ID. Arithmetic instructions and effective address calculations consume their operands in

Consumer	Producers	Purpose
ID RS1	EX, MEM, WB	Early branch/JR/JALR dependency resolution.
EX RS1/RS2	MEM, WB	Arithmetic, logic, shift, multiply, and address-generation operands.
MEM store data	WB	Supplies store data that is being written back in the same cycle.
Register-file read	WB write port	Same-cycle register-file write/read bypass.

Table 8: Forwarding paths in the processor.

EX. Store data consumes in MEM. Producers are classified as ordinary EX results, load/MEM results, or multiplier results.

For the current valid IF/ID instruction, the hazard unit checks all registers that the instruction actually uses and finds the latest availability among them. A RAW stall is generated only when this availability is later than the consumer stage. Dependencies that the forwarding network can satisfy are allowed to issue without an extra bubble.

The scoreboard is frozen during a data memory stall so that its view of the pipeline cannot advance while the datapath is held. A separate one-cycle reservation is also created after an issued multiply. This prevents an ordinary one-cycle EX instruction from trying to use the shared EX completion slot in the same cycle in which the delayed multiplication result emerges.

When a Decode hazard is detected, PC and IF/ID are held and the ID/EX validity bit is cleared for the younger issue slot. Older instructions are allowed to continue unless the entire pipeline is frozen by the data memory interface.

2.3.4 Branch and Jump Resolution

All supported branches and jumps are resolved in Decode. The branch unit uses the forwarded Decode value of RS1, the extended immediate, the current instruction PC, and the next address predicted by IF.

The six conditional branch instructions compare RS1 with zero. J and JAL are unconditional PC-relative transfers using the shifted 26 bit immediate, while JR and JALR take the target directly from RS1. The branch target is computed with a P4 adder as

$$PC_{target} = PC + 4 + (\text{signext}(imm) \ll 2). \quad (4)$$

The branch unit reconstructs the actual next PC and compares it with the next PC that was predicted in Fetch. If they differ, a correction/flush signal is sent directly back to IF. With a ready instruction memory, only the younger fetched instruction has to be discarded, giving a one-cycle correction penalty. The trade-off is that the branch decision and target path cross from Decode back to Fetch within one clock period.

2.4 Execute Stage

The Execute stage contains the arithmetic datapath. Before execution, RS1 and RS2 pass through forwarding muxes that can select data from MEM or WB. The second execution operand is then selected between the forwarded RS2 value and the decoded immediate.

The EX stage contains the arithmetic/comparison unit, logic unit, T2 shifter, pipelined multiplier, immediate pass-through path for LHI, and $PC + 4$ pass-through path used by JAL/JALR. Addition, subtraction, equality tests, and signed/unsigned relational comparisons share the arithmetic/comparison unit. The comparison functions reuse the subtraction/carry/sign information rather than instantiating independent full-width comparators for each instruction.

2.4.1 Two stage Radix-4 Booth–Dadda Multiplier

The multiplier is the main multi-cycle execution structure in the processor. It is a two stage pipelined 32×32 multiplier that returns the lower 32 bits of the product. The datapath consists of radix-4 Booth partial-product generation, a Dadda reduction tree, and a final P4 carry-propagate addition.

The Booth encoder examines overlapping three bit groups of the multiplier and generates the corresponding 0, $\pm A$, or $\pm 2A$ partial product. For the 32 bit operand width, 16 Booth groups are produced. An additional correction row carries the two’s-complement correction bits required by negative Booth terms, so 17 rows enter the Dadda tree.

The Dadda tree reduces the row count through

$$17 \rightarrow 13 \rightarrow 9 \rightarrow 6 \rightarrow 4 \mid 4 \rightarrow 3 \rightarrow 2, \quad (5)$$

where the vertical bar denotes the internal pipeline register. The first side of the register contains the first four reduction levels; the second side reduces the registered four rows to two. Those final sum and carry vectors are combined by the P4 adder.

The multiplier pipelines both a valid bit and the destination-register address with the arithmetic data. Its result therefore remains associated with the correct architectural instruction when it emerges later than a normal EX result. The structure is pipelined rather than iterative, so a new multiply may enter on the following cycle when the global pipeline is not stalled. The EX output logic gives priority to a valid multiplier completion, while the hazard unit reserves the conflicting normal-EX completion slot.

2.4.2 P4 Adder

The P4 adder is the common fast adder reused in the main arithmetic unit, branch target calculation, and multiplier final addition. The 32 bit design combines a sparse carry generation tree with 4 bit carry select sum blocks.

The carry generator first forms bitwise propagate and generate terms. Local prefix combinations are then used to obtain the carry at each 4 bit block boundary. The sum generator contains 4 bit carry-select blocks: each block computes candidate sums for carry-in zero and carry-in one, and the sparse-tree carry selects the correct result. This limits the long carry-dependent path without requiring a fully dense prefix tree across every sum bit.

2.5 Memory Stage

The Memory stage receives either an ordinary EX result or the effective address of a load/store operation. The pipelined memory controls determine whether the stage accesses data memory or simply forwards the EX result toward WB.

LW and SW are forced to a word-aligned address by clearing the two least significant address bits. Byte accesses retain the original byte offset. The data path uses big-endian byte ordering. For LB and LBU, the effective address offset selects bits [31 : 24], [23 : 16], [15 : 8], or [7 : 0] from the returned 32 bit word. LB sign extends the selected byte and LBU zero extends it.

For SB, the low byte of the source operand is supplied together with the byte store qualifier and the original byte offset. A WB-to-MEM forwarding path is available for store data, allowing a store to use a value that is being written back in the same cycle.

The MEM forwarding interface does not advertise an incomplete load as an ordinary MEM result. ALU-type results are available for forwarding from MEM, while a load becomes a producer through the WB path after the returned memory data has been captured.

Multiplier completions pass through MEM as ordinary execution results. When the EX/MEM entry is marked as a multiply result, memory read/write controls are suppressed so that the delayed multiply completion cannot be interpreted as a memory operation.

2.5.1 External Memory Interface

The processor does not assume a fixed memory latency. Instruction and data memory are exposed through explicit `issue/ready` handshakes. The memory-interface block sits between the internal pipeline and the external instruction/data ports.

On the instruction side, the processor keeps the instruction request active while out of reset. If `iram_ready` is not asserted, the interface raises the instruction memory stall and Fetch retains the current request.

On the data side, `dram_issue` is asserted only for an active load or store. A request for which `dram_ready=0` produces a data memory stall. This stall freezes the pipeline registers, the pipelined control state, and the hazard scoreboard so that the pending memory instruction and every younger instruction remain aligned until the transaction completes.

For a read, the processor leaves the bidirectional data bus undriven and samples the returned data when the memory responds. For a write, the processor drives the write data and deasserts `dram_readnotwrite`. The `dram_store_byte` output distinguishes SB from a full-word store.

Keeping this handshake logic in a dedicated top-level interface allows IF and MEM to express memory operations in pipeline terms while the interface handles external request timing and backpressure.

2.6 Writeback Stage

The WB stage commits the MEM/WB value to the register file when the pipeline entry and destination are valid. The normal decoded writeback enable handles ordinary instructions, while the delayed multiplier validity is carried with the result so that a multiplication completion is also committed correctly. Writes to R0 remain suppressed in the register file.

The WB destination address and value are also exported to the forwarding network. They can feed EX operands, the Decode branch/jump bypass, MEM store data, and the register-file same-cycle bypass. During a data memory stall, WB writeback is held consistently with the rest of the pipeline so that an instruction is not committed repeatedly while the memory transaction is pending.

3 Functional Verification

Functional verification was performed using directed assembly programs executed on the RTL model in Questa/ModelSim. The assembly-to-memory-image tool used as the starting point was provided by the university and was modified to support the implemented ISA and its encoding choices. For each test, the assembly source was converted into the instruction memory image and the initial data memory image used by the simulation.

3.1 Verification Environment

The processor was instantiated together with separate instruction and data memory models. The testbench loads the generated instruction and data images through file path generics and runs the processor until the program writes the completion value to a completion address. At the end of simulation, the resulting data memory is dumped to a file.

A Python script automates assembly, RTL compilation, simulation, and result checking. The generated data memory dump is compared directly against the expected memory image associated with the test program. Exact memory image equivalence is used as the primary pass/fail condition, while waveform files are retained for debugging when required.

3.2 Directed Test Programs

The stimuli program set was chosen to exercise both individual ISA groups and pipeline interactions. The main coverage of the supplied stimuli is summarized in Table 9.

Test category	Main behavior exercised
Arithmetic and register behavior	Signed/unsigned arithmetic, negative immediates, and constant zero register behavior
Logic and shifts	Logical operations, immediate zero extension, logical/arithmetic shifts, and shift counts
Comparisons	Signed and unsigned register/immediate comparisons
Branches and control flow	Conditional branches, branch flushing, J, JAL, JR, JALR, and return behavior
Memory operations	Word and byte accesses, signed/unsigned byte loads, store byte behavior, and big endian byte placement
Multiplication	Pipelined multiplication, dependent operations, truncation to the low 32 bits, and looped multiplication
Integrated workloads	Array dot product, address generation, loops, and combined arithmetic/memory operation
Pipeline dependencies	Load use, ALU chains, load to store, multiply use, branch use, forwarding, and hazard handling without manually inserted dependency NOPs

Table 9: Directed RTL verification stimuli.

3.3 RTL Verification Result

The RTL implementation passed the complete directed regression: for every verification program, the final data memory dump matched its expected memory image. This verified the implemented instruction behavior together with the forwarding, hazard handling, control flow recovery, multiplier dependencies, and memory interface behavior exercised by the stimuli program set.

4 Synthesis Flow and Methodology

The synthesis phase was treated as a design space exploration problem rather than as a single compile performed at one chosen clock period. The main objective was to determine how the synthesis timing constraint affected the resulting mapped implementation, identify functionally valid mappings, and compare those mappings at operating points derived from their *actual* achieved timing. In addition to the baseline design, several clock-gating configurations were explored in order to evaluate the timing, area, and power trade offs introduced by clock gating insertion.

4.1 Timing Driven Synthesis Exploration

Initial synthesis experiments showed that the range of interest was between 0.7 ns and 1.8 ns. A clock constraint sweep was then performed. Independent mappings were generated at a sequence of synthesis constraints in this range for the baseline processor and for the investigated clock gating configurations.

The sweep showed that mapping quality was not proportional with the requested clock period. Tightening the constraint generally caused Design Compiler to apply more aggressive timing optimization, but after certain point smaller synthesis periods did not continuously produce smaller critical path delays. Instead, the achieved delay improved up to a certain optimization point and could then become worse under still tighter constraints.

For example, with the aggressive end of the sweep, the mapping generated with a 0.8 ns synthesis constraint yielded better timing performance than the mapping generated at 0.7 ns, even though 0.7 ns was the more stringent constraint. The observed behavior is consistent with the fact that timing driven synthesis is a heuristic optimization process.

4.2 Synthesis Constraint and Achieved Operating Period

A second important part of the methodology was to separate the clock period used to *drive synthesis* from the clock period that the resulting mapping could actually support. Aggressively constrained mappings failed to meet the period with which they had been synthesized. The aggressive period was used to push the synthesis optimization, not as the expected final operating period.

For every generated mapping, the achieved critical path delay T_{critical} was extracted. A separate analysis period was then defined as

$$T_{\text{analysis}} = 1.05 T_{\text{critical}}, \quad (6)$$

providing approximately 5% timing margin relative to the measured critical path. The corresponding analysis frequency was

$$f_{\text{analysis}} = \frac{1}{T_{\text{analysis}}}. \quad (7)$$

The three quantities used throughout the synthesis study are:

1. the **synthesis period**, which controls the timing constraint applied during mapping;
2. the **achieved critical path delay**, which characterizes the timing quality of the generated implementation; and
3. the **analysis period**, defined as the critical path delay plus the 5% margin and used for subsequent timing and power analysis.

The design was *not* re synthesized at T_{analysis} . The generated mapped design was reloaded and reanalyzed using the new period.

4.3 Clock Sweep Procedure

For each synthesis target T_i , the following procedure was applied:

1. The RTL was synthesized using T_i as the timing constraint, producing a mapped implementation M_i .
2. The critical path delay $T_{\text{critical},i}$ was extracted from that mapped implementation.
3. An analysis period was assigned to the mapping by adding the 5% timing margin to its measured critical path, following Equation 6.
4. The same mapping M_i was then reloaded, and analyzed at $T_{\text{analysis},i}$ to obtain its timing, area, and activity aware power results.

This procedure allowed implementations produced under substantially different synthesis pressures to be compared at operating points derived from the timing actually achieved by each mapping, while preserving the implementation created by the original synthesis run.

4.4 Clock Gating Design Space Exploration

Clock gating was explored in addition to the baseline design. The synthesis runs varied the minimum bit width setting used for clock gating insertion, with configurations corresponding to minimum widths of 1, 8, 16, and 32 bits. Each clock-gating family was subjected to the same timing sweep and fixed mapping analysis procedure as the baseline design.

The sweep varied both the synthesis clock constraint and the clock gating granularity. Each resulting mapping was then compared using timing, area, power, and functional verification.

4.5 Post-Synthesis Functional Simulation

The most aggressive mappings showed an additional constraint on the design space. The mappings synthesized at 0.7 ns and 0.8 ns failed the required post-synthesis mapped-netlist functional verification. This behavior was observed for the baseline mapping and for the investigated clock gating families at those synthesis points.

Consequently, timing, area, and power were not sufficient selection criteria on their own. A mapping was retained as a candidate only if it also passed the post synthesis functional verification. The 0.7 ns and 0.8 ns mappings were excluded from the subsequent PPA comparison, even though the 0.8 ns synthesis run represented the best raw timing result seen for the tightest synthesis constraints.

4.6 Activity Aware Power Analysis

After functional filtering, every surviving mapping was reanalyzed at its own T_{analysis} operating point. Timing and area reports were collected from the fixed mapped design, while switching activity obtained from simulation was back-annotated for power estimation. The simulation stimulus was chosen as an average workload, intended to exercise the processor with a representative mix of activity.

The resulting dataset therefore associates each valid implementation with an operating period derived from its own achieved timing and with power evaluated using simulation activity.

4.7 Valid Mapping Results

Table 10 summarizes the functionally valid mappings retained after the 0.7 ns and 0.8 ns points were removed. The period shown is the analysis/operating period used for the fixed mapping. Dynamic power is the sum of the reported internal and switching components, and total power additionally includes leakage.

Mapping	Period (ns)	MHz	Int. (mW)	Sw. (mW)	Dyn. (mW)	Leak (mW)	Total (mW)	Area	Setup (ns)	Hold (ns)
Baseline 0.9	1.428	700.280	15.20	3.41	18.61	0.707	19.317	33909.4	0.06	0.07
Baseline 1.0	1.428	700.280	14.60	3.49	18.09	0.686	18.776	32829.2	0.07	0.07
Baseline 1.4	1.470	680.272	14.80	3.32	18.12	0.623	18.743	30656.2	0.07	0.06
Baseline 1.5	1.575	634.921	12.90	3.05	15.95	0.598	16.548	29438.2	0.08	0.09
Baseline 1.2	1.649	606.428	12.30	2.98	15.28	0.650	15.930	31288.5	0.08	0.09
Baseline 1.6	1.680	595.238	12.20	2.87	15.07	0.599	15.669	29495.9	0.08	0.08
CG32 1.2	1.796	556.793	4.84	3.03	7.87	0.487	8.357	24352.6	0.00	0.05
CG32 1.6	1.817	550.358	4.80	3.03	7.83	0.484	8.314	24211.6	0.00	0.05
CG8 0.9	1.817	550.358	4.85	3.11	7.96	0.512	8.472	25547.2	0.00	0.05
CG1 1.2	1.838	544.070	4.56	2.88	7.44	0.482	7.922	24365.3	0.00	0.05
CG1 1.0	1.859	537.924	4.54	2.86	7.40	0.497	7.897	24981.9	0.00	0.05
CG8 1.0	1.859	537.924	4.75	3.01	7.76	0.507	8.267	25095.2	0.00	0.05
CG1 0.9	1.869	535.045	4.53	2.84	7.37	0.494	7.864	24867.8	0.00	0.05
CG1 1.6	1.880	531.915	4.44	2.82	7.26	0.478	7.738	24175.4	0.00	0.05
Baseline 1.8	1.890	529.101	10.60	2.49	13.09	0.583	13.673	28964.7	0.09	0.06
CG32 1.4	1.890	529.101	4.63	2.89	7.52	0.485	8.005	24287.7	0.00	0.05
CG32 1.8	1.890	529.101	4.56	2.87	7.43	0.471	7.901	23850.9	0.00	0.05
CG16 0.9	1.901	526.039	4.63	2.90	7.53	0.504	8.034	25273.2	0.00	0.05
CG16 1.2	1.901	526.039	4.59	2.85	7.44	0.481	7.921	24208.4	0.00	0.05
CG32 0.9	1.901	526.039	4.63	2.89	7.52	0.529	8.049	25802.0	0.00	0.05
CG32 1.0	1.901	526.039	4.59	2.87	7.46	0.501	7.961	25047.6	0.00	0.05
CG16 1.5	1.911	523.286	4.53	2.86	7.39	0.483	7.873	24117.2	0.00	0.05
CG1 1.4	1.911	523.286	4.36	2.76	7.12	0.478	7.598	24131.3	0.00	0.05
CG1 1.8	1.911	523.286	4.38	2.79	7.17	0.476	7.646	24146.1	0.00	0.05
CG8 1.2	1.911	523.286	4.51	2.86	7.37	0.487	7.857	24513.5	0.00	0.05
CG8 1.4	1.911	523.286	4.46	2.86	7.32	0.475	7.795	24070.9	0.00	0.05
CG16 1.0	1.932	517.598	4.55	2.83	7.38	0.497	7.877	24800.2	0.00	0.05
CG16 1.8	1.943	514.668	4.46	2.82	7.28	0.477	7.757	24027.8	0.00	0.05
CG1 1.5	1.943	514.668	4.30	2.71	7.01	0.475	7.485	24045.9	0.00	0.05
CG32 1.5	1.943	514.668	4.54	2.81	7.35	0.474	7.824	24009.7	0.00	0.05
CG8 1.6	1.953	512.033	4.38	2.83	7.21	0.477	7.687	24001.7	0.00	0.05
CG16 1.6	1.974	506.586	4.38	2.78	7.16	0.477	7.637	23987.6	0.00	0.05
CG16 1.4	2.027	493.340	4.26	2.67	6.93	0.481	7.411	24165.0	0.00	0.05
CG8 1.5	2.048	488.281	4.13	2.65	6.78	0.470	7.250	23883.9	0.00	0.05
CG8 1.8	2.048	488.281	4.13	2.65	6.78	0.474	7.254	23967.1	0.00	0.05

Table 10: Timing, power, and area results for functionally valid synthesis mappings. “CG w ” denotes a clock-gated mapping with minimum clock-gating bit-width w ; the final number is the synthesis target in ns.

4.8 Final Mapping Selection

After functional filtering and comparison of the remaining timing, power, and area results, the implementation selected for the physical design flow was

`clock_gated_w32_sweep_1.2ns.`

Its 1.2 ns value is the *synthesis optimization target*; it is not the final operating clock period. For the fixed mapped implementation, the selected analysis period was approximately

$$T_{\text{analysis}} \approx 1.796 \text{ ns}, \quad (8)$$

which corresponds to

$$f_{\text{analysis}} \approx \frac{1}{1.796 \text{ ns}} \approx 556.8 \text{ MHz}. \quad (9)$$

The selected `clock_gated_w32_sweep_1.2ns` mapping provided the best overall trade off among the functionally valid candidates. At its analysis period of 1.796 ns, it achieved approximately 7.87 mW dynamic power, 8.357 mW total power, and an area of 24352.6. Other clock-gated mappings reduced power or area slightly further, but only at lower operating frequencies, while the baseline implementations were consistently more expensive in both power and area. For example, the baseline synthesized at the same 1.2 ns target required a 1.649 ns analysis period and reported 15.930 mW total power with area 31288.5. The selected mapping was therefore chosen as the most balanced performance-oriented solution rather than the absolute minimum-power, minimum-area, or fastest implementation.

4.9 Methodological Summary

The synthesis methodology can be summarized as follows:

- Explore the useful range of synthesis timing constraints;
- Generate independent mappings across the selected clock sweep;
- Repeat the sweep for the baseline and clock-gating configurations;
- Extract the achieved critical path of each mapping;
- Perform post-synthesis functional verification and reject mappings that fail the regression;
- Reload each surviving mapping without re-synthesizing it;
- Set the analysis period to $1.05 T_{\text{critical}}$;
- Back-annotate simulation switching activity for power analysis;
- Collect timing, area, and power from the fixed mapping; and
- Select the implementation to be carried into physical design from the resulting functionally valid PPA design space.

Two observations justify the design-space approach particularly clearly. First, the achieved timing was non-monotonic with synthesis pressure, so the most stringent requested period did not necessarily produce the fastest mapping. Second, the numerically fastest region of the sweep did not satisfy the post-synthesis verification requirement. A single synthesis run would have hidden both effects and would not have provided a defensible basis for the final implementation choice.

5 Physical Design

The selected netlist was implemented using **Cadence Innovus** with the **Nangate45/FreePDK45** technology. The implementation used the 1.796 ns operating period selected after synthesis, corresponding to approximately 556.8 MHz.

5.1 Implementation Flow

- import the synthesized netlist and timing constraints;
- create the floorplan and power distribution network;
- perform placement and clock-tree synthesis (CTS);
- route the design;
- run post-route setup and hold optimization; and
- check timing, power, connectivity, and design rule violations.

The floorplan was square with an initial utilization target of 60%. VDD and ground rings and stripes were used for power distribution, while signal routing used metal layers M1–M6.

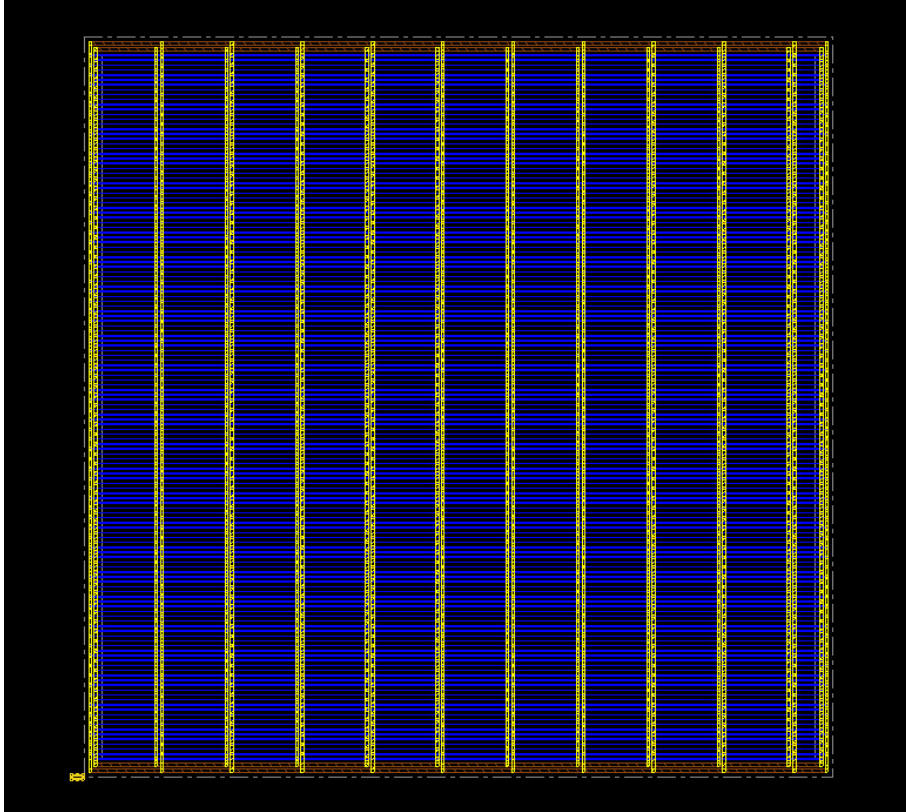


Figure 4: Floorplan with power distribution

Placement and CTS was performed with a target clock transition of 50 ps and a target skew of 20 ps, followed by routing and post-route setup/hold optimization.

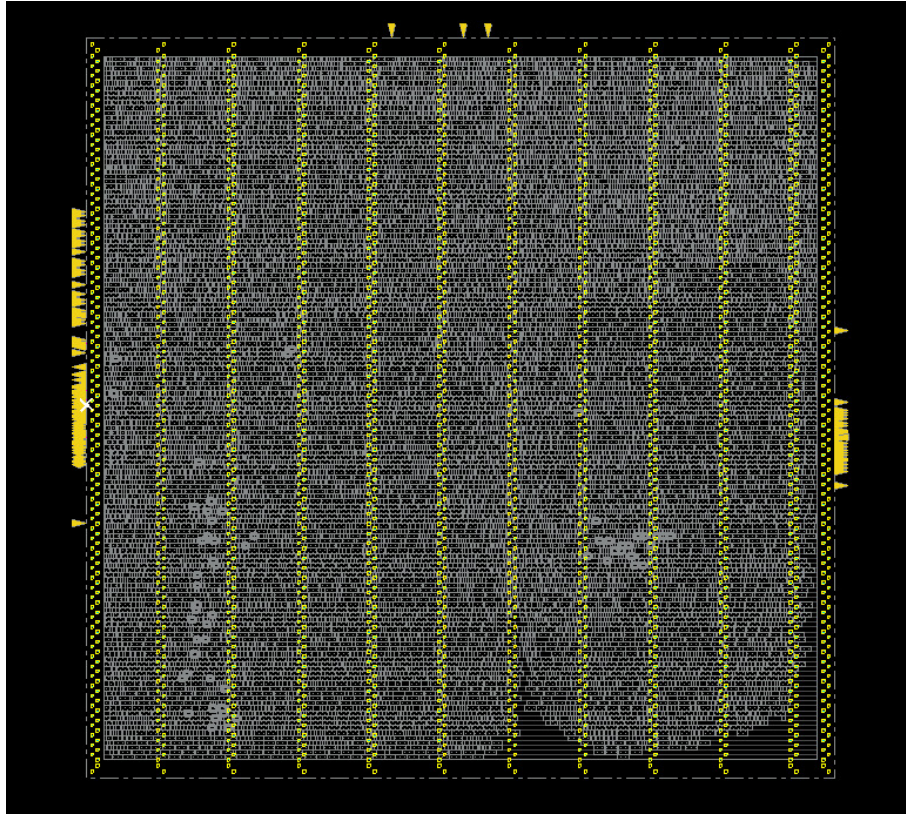


Figure 5: Cell placement

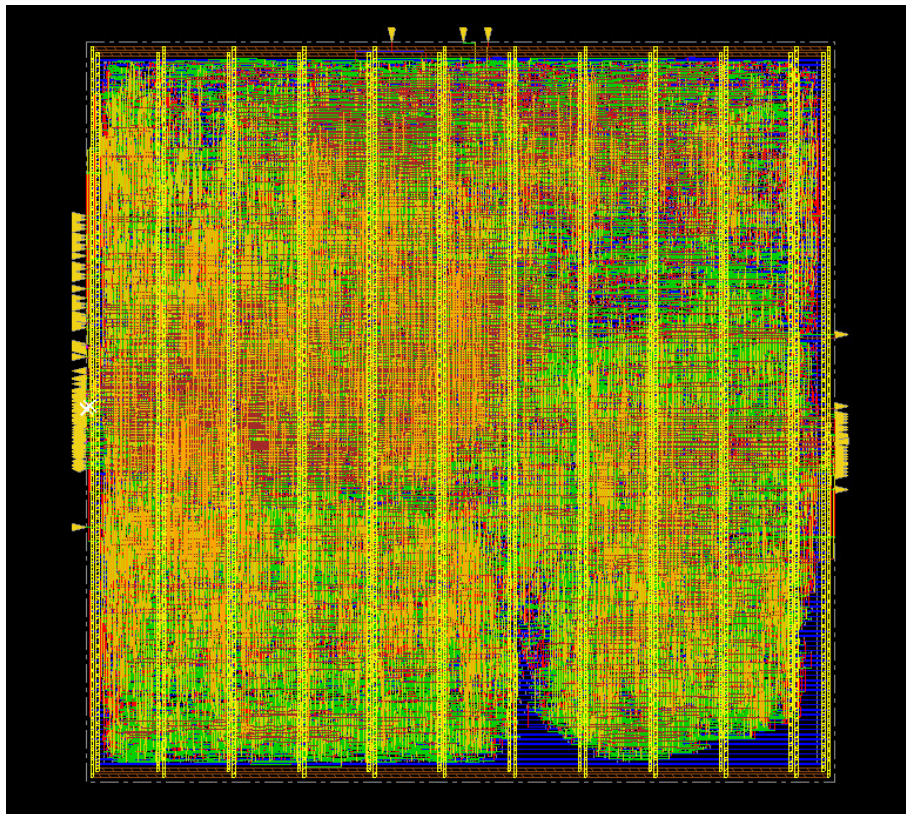


Figure 6: Routing and Clock tree Synthesis

5.2 Timing Analysis

Post route timing was analyzed at typical, slow, and fast corners. For the scope of the academic project, timing closure was accepted at the **typical corner**. At 556.8 MHz, the design achieved **+103 ps setup slack** and **+18 ps hold slack**, with no setup or hold violations under the applied constraints. External interface timing was not specified.

Closure was not claimed for all the corners. The slow corner reported a setup WNS of -300 ps, while the fast corner reported a hold WNS of -33 ps.

5.3 Post Route Results

The final implementation contained **22,343 instances**. The main physical metrics are summarized in Table 11.

Metric	Result
Standard-cell area	$36,647.884 \mu\text{m}^2$
Core area	$40,624.584 \mu\text{m}^2$
Die area	$44,827.490 \mu\text{m}^2$
Final utilization	90.211%

Table 11: Post-route physical implementation metrics.

Typical corner post-route power at 1.1 V was evaluated using updated activity assumptions. The reported power components are shown in Table 12.

Power component	Power
Internal	11.636 mW
Switching	10.546 mW
Leakage	0.925 mW
Total	23.107 mW
Clock network (included in total)	2.402 mW

Table 12: Typical-corner post-route power estimate using the updated activity assumptions.

The updated total power is approximately 24.8% lower than the earlier 30.712 mW vectorless estimate. This difference reflects the change in activity assumptions.

Connectivity verification reported no problems, and no electrical design rule violations were reported. Four physical DRC violations remained, including one metal short.